Face Attendance Project Guide

Simple pointers for explaining the attendance project

1. What the project does

- Keeps an electronic attendance register.
- An administrator registers students and their face photos.
- A faculty member signs in and opens the classroom camera kiosk.
- The system checks a student's face against registered students and records a match.
- Students do not need an account. They only stand in front of the faculty camera.

2. Who can do what

- **Admin:** registers students; views and exports attendance; manages settings and email reports.
- **Faculty:** signs in and uses the attendance kiosk only.
- **Student:** does not sign in and cannot edit records; the student is recognized at the kiosk.
- The server enforces these role limits, even if someone tries to open an admin address directly.

3. How attendance is recorded

1. Faculty opens the site and signs in.
2. Faculty allows the browser to use the camera.
3. The browser sends camera frames to the app server over HTTPS.
4. The face model looks for a face and makes a numerical face signature.
5. The system compares that signature with the signatures stored for registered students.
6. When the closest match passes the configured threshold, the system records attendance.
7. Faculty sees the result. Admin can review the attendance list or download it as a CSV file.

4. Login and logout

- Open the login page.
- Enter the Admin or Faculty username and password provided by the project owner.
- A successful sign-in creates a secure, signed session that lasts up to eight hours.
- Choose **Log out** in the dashboard to end the session.
- Keep passwords private. Never put real passwords in this public repository.

5. Simple architecture

- **Browser:** shows the pages and asks for camera permission.
- **App server:** checks the account role, receives camera frames, and coordinates the work.

- **Face model:** detects and compares faces on the server.
- **Database:** keeps student and attendance records.
- **Files:** keep registered face photos, numerical face signatures, and settings.
- **Optional email service:** can send reports after an admin configures email settings.

In one line: **Camera -> Browser -> App server -> Face model -> Attendance database -> Dashboard**.

6. The face model on the live demo

- The live Render demo uses the InsightFace `buffalo_sc` face detection and recognition pack.
- The smaller pack is about 16 MB and successfully loaded on the free service.
- The app uses CPU processing; the browser provides the camera, while the server runs recognition.
- The model can make mistakes. Lighting, camera quality, pose, and registration photos affect results.
- The project does not check that a live person is present; a photo or screen may fool it.
- InsightFace's pretrained model packs are limited to non-commercial research use. Check the license before any other use.

7. Project file pointers

- `web_app.py` - main server; login sessions, role checks, model startup, API routes, attendance, exports, and email.
- `static/login.html` - sign-in page for Admin and Faculty.
- `static/index.html` - dashboard page structure.
- `static/app.js` - browser actions, camera frames, recognition loop, and dashboard updates.
- `static/style.css` - dashboard layout, colors, and faculty view styling.
- `attendance_system.py` - earlier command-line version; the website uses `web_app.py`.
- `run_web.py` - starts the web app for local use.
- `run_online.py` - starts a temporary public tunnel for a short demo.
- `requirements.txt` - Python packages needed by the app.
- `Dockerfile` - instructions to package the app for a container host.
- `docker-compose.yml` - runs the container locally and keeps local data in a named volume.
- `docker-entrypoint.sh` - prepares the data folder when the container starts.
- `render.yaml` - Render service configuration, including the lightweight model setting.
- `README.md` - short overview and local setup steps.
- `DEPLOYMENT.md` - hosting notes and limitations.
- `QUICK_TUNNEL.md` - temporary tunnel instructions; the computer must stay on.
- `REPORT_REVIEW.md` - notes on the original report and implementation differences.
- `output/pdf/PROJECT_GUIDE.pdf` - this guide in PDF form.
- `.env.example` - names of settings to configure locally; it contains placeholders, not real credentials.

8. Live link and important free-plan limits

- Website: minor-project-attendance.onrender.com
- Sign in: minor-project-attendance.onrender.com/login
- Source code: github.com/tiwari-shresth/Minor-Project
- The Render free service has limited memory and temporary storage.
- Free-service attendance data, photos, and face signatures may disappear after a restart or redeploy.
- It can sleep when unused, so opening it may take a little time.
- Use sample data for a demo. Do not depend on this free service for official attendance or real biometric records.
- For real use, arrange persistent protected storage, backups, a retention and deletion policy, suitable consent, and an attendance alternative.

9. Short explanation to present

“The admin registers students first. When faculty starts a class, they sign in and open the camera kiosk. The system compares each student's face with the registered list and records a match in the attendance register. Students do not need accounts, and only the admin can manage records.”